Exp No.: 8 Fabric Network Deployment and Management

Aim

To deploy, manage, monitor, and maintain a Hyperledger Fabric network by creating the network, verifying network components, deploying chaincode, monitoring Docker containers, and performing network shutdown and cleanup operations.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Hyperledger Fabric network deployment involves setting up the blockchain infrastructure required for enterprise applications. A Fabric network consists of peer nodes, ordering service, Certificate Authorities (CAs), channels, chaincode, and client applications. Network management includes:

- Starting and stopping the network
- Creating application channels
- Deploying chaincode
- Monitoring Docker containers
- Managing peer and orderer services
- Viewing network logs
- Cleaning up Docker containers and generated artifacts

Proper deployment and management ensure reliable blockchain operations and simplify maintenance and troubleshooting.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network
./network.sh up createChannel -ca

Expected Output
Creating channel 'mychannel'

Starting Certificate Authorities

Starting peer0.org1

Starting peer0.org2

Starting orderer.example.com

Fabric Network Started Successfully

Step 3: Verify Running Docker Containers
Display all active Docker containers.
docker ps

Expected Output
peer0.org1.example.com

peer0.org2.example.com

orderer.example.com

ca_org1

ca_org2

Step 4: Display Docker Images
docker images

Expected Output
hyperledger/fabric-peer

hyperledger/fabric-orderer

hyperledger/fabric-ca

hyperledger/fabric-tools

Step 5: Check Network Status
Display all Docker containers including stopped containers.
docker ps -a

Expected Output
STATUS

Up 3 minutes

Running

Step 6: Deploy Asset Transfer Chaincode
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
Expected Output
Packaging Chaincode

Installing Chaincode

Approving Chaincode

Committing Chaincode

Chaincode deployed successfully

Step 7: Verify Installed Chaincode
peer lifecycle chaincode queryinstalled

Expected Output
Package ID:

basic_1.0

Version:1.0

Step 8: Monitor Peer Logs
Display logs of the peer node.
docker logs peer0.org1.example.com

Expected Output
Peer started successfully

Listening on port 7051

Chaincode installed successfully

Step 9: Monitor Orderer Logs
docker logs orderer.example.com

Expected Output
Orderer started

Channel mychannel initialized

Transactions processed successfully

Step 10: Verify the Channel
peer channel list

Expected Output
Channels peers has joined:

mychannel

Step 11: Stop the Fabric Network
./network.sh down

Expected Output
Removing containers

Removing Docker Network

Network stopped successfully

Step 12: Clean Docker Resources (Optional)
Remove unused Docker containers, networks, and images.
docker system prune -f

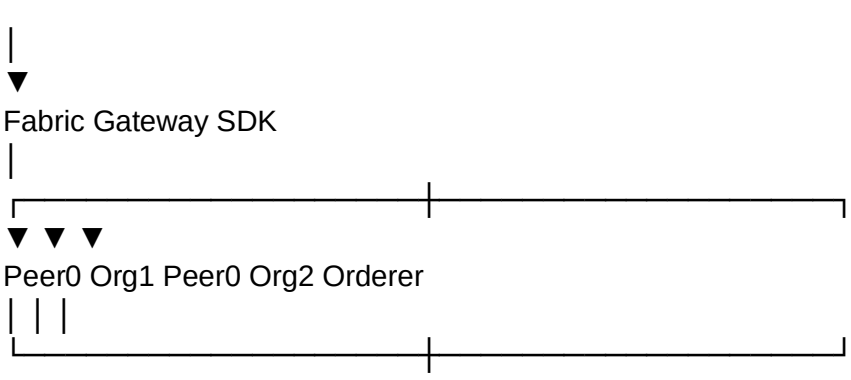
Expected Output
Deleted Containers

Deleted Networks

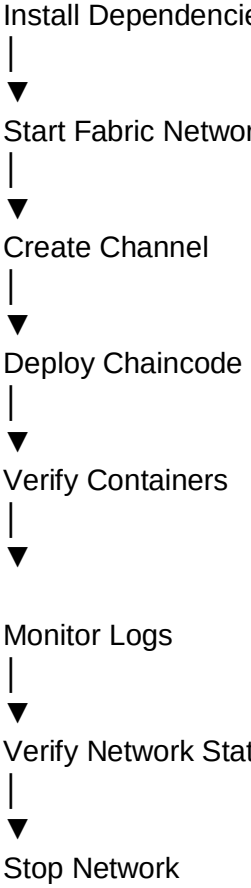
Deleted Images

Total reclaimed space: xxx MB

Architecture Diagram
Hyperledger Fabric Network



Network Management Workflow



|
▼
Clean Resources

Sample Output
Fabric Network Started Successfully

Docker Containers Running

Peer Nodes Active

Orderer Active

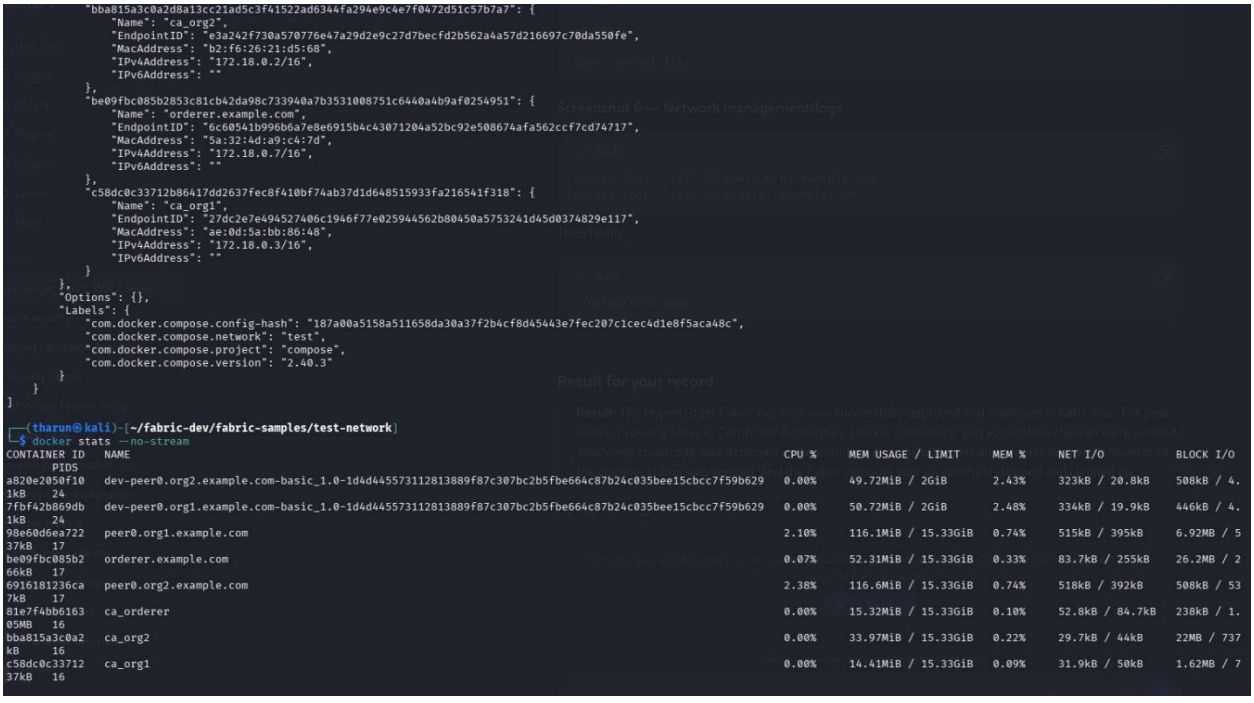
Certificate Authorities Running

Channel Created Successfully

Chaincode Installed Successfully

Network Verified Successfully

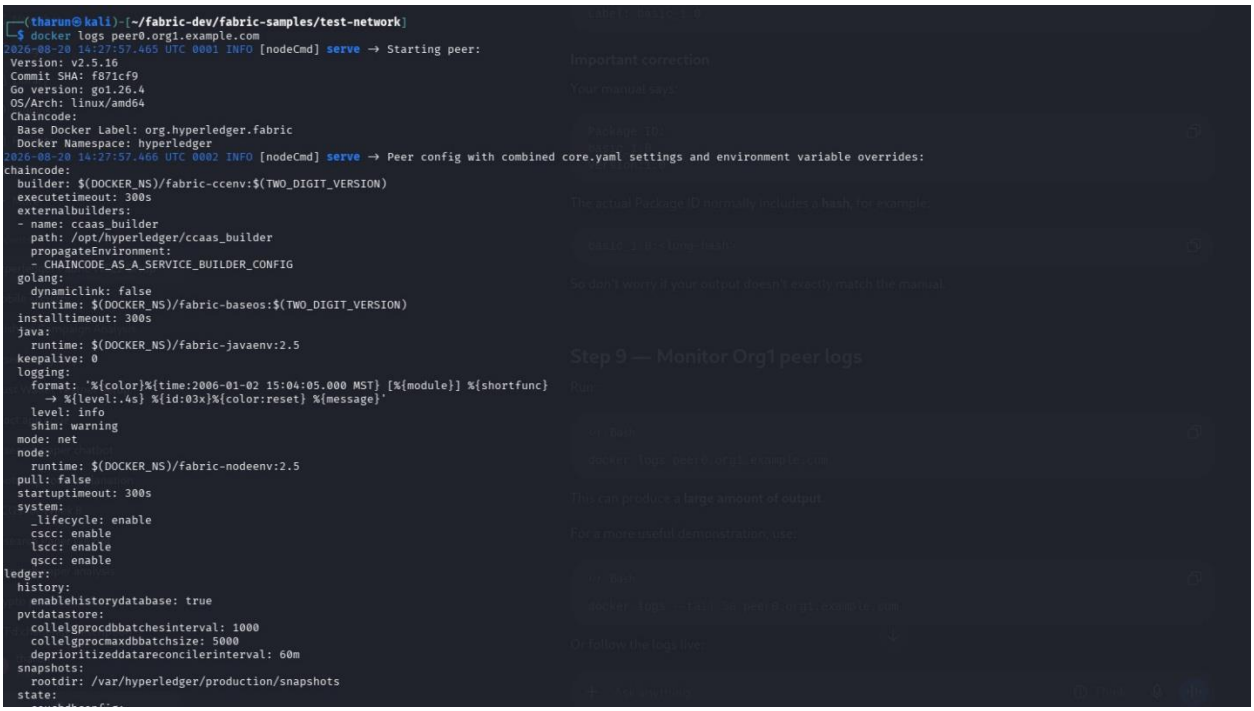
Fabric Network Stopped Successfully



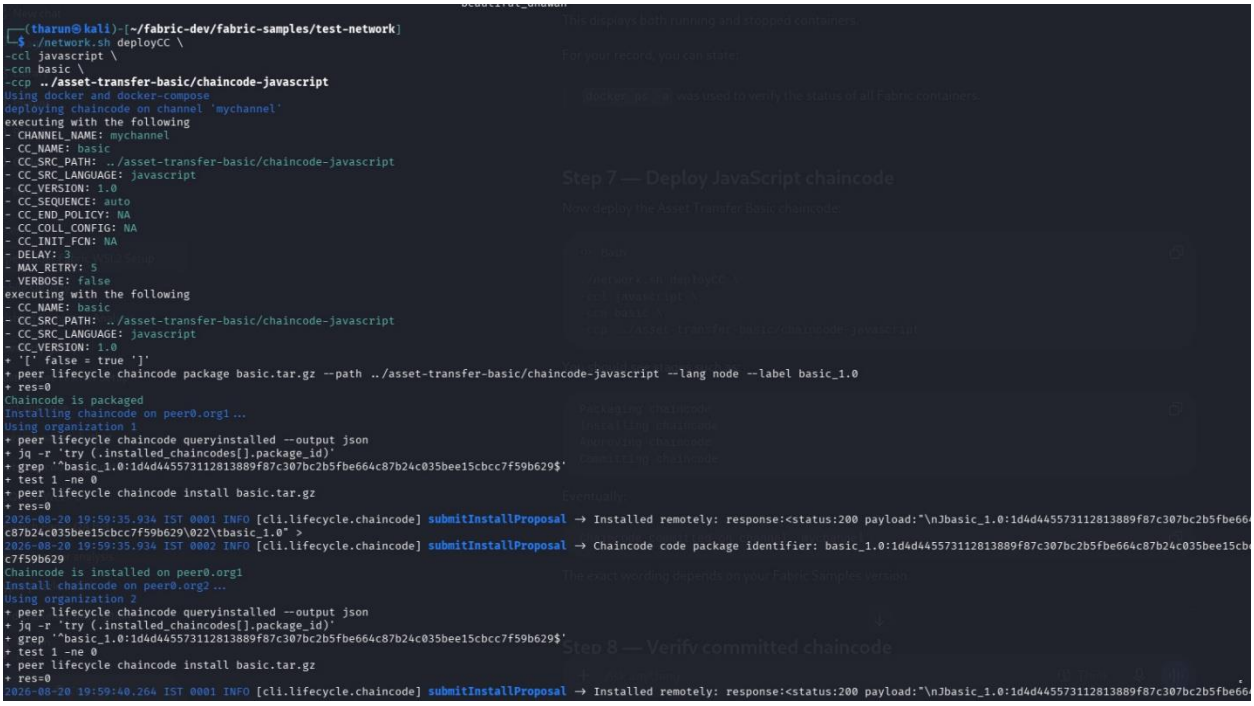
```
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ docker network ls
NETWORK ID     NAME          DRIVER    SCOPE
f03bc7acdf85   bridge        bridge    local
b61941ea85be   fabric_test   bridge    local
a930fe55c95b   host          host      local
c17e6ebb3833   none          null      local

┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ docker network inspect fabric_test
[
    {
        "Name": "fabric_test",
        "Id": "b61941ea85be2843e61e6a315584a64aa136eed6a74a837ed49a3c1e3e7a6ce7",
        "Created": "2026-08-20T19:57:51.61299705+05:30",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "6916181236caf32e03aed60bf9b1c8a5a433e224973f9b778ed19ba9f52774b7": {
                "Name": "peer0.org2.example.com",
                "EndpointID": "cc09c02c3011556f45f25274a88512467cfa2d6ef61ec3ddedabecb52bbc5281",
                "MacAddress": "4e:58:27:48:24:66",
                "IPv4Address": "172.18.0.6/16",
                "IPv6Address": ""
            },
            "7fbf42b869dbed00a7e22b715b471f4f8cd83702444f80efee35cfa5352497fe": {
                "Name": "dev-peer0.org1.example.com-basic_1.0-1d4d44557312813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629",
                "EndpointID": "e59849a15fb9e1aa8897eb1b457675e267f19e700ea1c2e7d182242f65bea437",
                "MacAddress": "16:81:1a:52:60:61",
                "IPv4Address": "172.18.0.9/16",
                "IPv6Address": ""
            },
```

```
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ docker logs --tail 50 orderer.example.com
2026-08-20 14:28:00.757 UTC 0017 INFO [orderer.common.cluster] Configure → Exiting
2026-08-20 14:28:00.757 UTC 0018 INFO [orderer.consensus.etcdraft] start → Starting raft node as part of a new channel channel=mychannel node=1
2026-08-20 14:28:00.757 UTC 0019 INFO [orderer.consensus.etcdraft] switchToConfig → 1 switched to configuration voters=() channel=mychannel node=1
2026-08-20 14:28:00.757 UTC 001a INFO [orderer.consensus.etcdraft] becomeFollower → 1 became follower at term 0 channel=mychannel node=1
2026-08-20 14:28:00.757 UTC 001b INFO [orderer.consensus.etcdraft] newRaft → newRaft 1 [peers: [], term: 0, commit: 0, applied: 0, lastindex: 0, lastterm: 0] channel=mychannel node=1
2026-08-20 14:28:00.757 UTC 001c INFO [orderer.consensus.etcdraft] becomeFollower → 1 became follower at term 1 channel=mychannel node=1
2026-08-20 14:28:00.758 UTC 001d INFO [orderer.consensus.etcdraft] switchToConfig → 1 switched to configuration voters=(1) channel=mychannel node=1
2026-08-20 14:28:00.758 UTC 001e INFO [orderer.consensus.etcdraft] run → This node is picked to start campaign channel=mychannel node=1
2026-08-20 14:28:00.759 UTC 001f INFO [orderer.consensus.etcdraft] switchToConfig → 1 switched to configuration voters=(1) channel=mychannel node=1
2026-08-20 14:28:00.759 UTC 0020 INFO [orderer.consensus.etcdraft] apply → Applied config change to add node 1, current nodes in channel: [1] channel=mychannel node=1
2026-08-20 14:28:01.758 UTC 0021 INFO [orderer.consensus.etcdraft] hup → 1 is starting a new election at term 1 channel=mychannel node=1
2026-08-20 14:28:01.758 UTC 0022 INFO [orderer.consensus.etcdraft] becomePreCandidate → 1 became pre-candidate at term 1 channel=mychannel node=1
2026-08-20 14:28:01.758 UTC 0023 INFO [orderer.consensus.etcdraft] poll → 1 received MsgPreVoteResp from 1 at term 1 channel=mychannel node=1
2026-08-20 14:28:01.758 UTC 0024 INFO [orderer.consensus.etcdraft] stepCandidate → 1 has received 1 MsgPreVoteResp votes and 0 vote rejections channel=mychannel node=1
2026-08-20 14:28:01.758 UTC 0025 INFO [orderer.consensus.etcdraft] becomeCandidate → 1 became candidate at term 2 channel=mychannel node=1
2026-08-20 14:28:01.762 UTC 0026 INFO [orderer.consensus.etcdraft] poll → 1 received MsgVoteResp from 1 at term 2 channel=mychannel node=1
2026-08-20 14:28:01.762 UTC 0027 INFO [orderer.consensus.etcdraft] stepCandidate → 1 has received 1 MsgVoteResp votes and 0 vote rejections channel=mychannel node=1
2026-08-20 14:28:01.762 UTC 0028 INFO [orderer.consensus.etcdraft] becomeLeader → 1 became leader at term 2 channel=mychannel node=1
2026-08-20 14:28:01.762 UTC 0029 INFO [orderer.consensus.etcdraft] run → raft.node: 1 elected leader 1 at term 2 channel=mychannel node=1
2026-08-20 14:28:01.763 UTC 002a INFO [orderer.consensus.etcdraft] run → Leader 1 is present, quit campaign channel=mychannel node=1
2026-08-20 14:28:01.763 UTC 002b INFO [orderer.consensus.etcdraft] run → Raft leader changed: 0 → 1 channel=mychannel node=1
2026-08-20 14:28:01.763 UTC 002c INFO [orderer.consensus.etcdraft] run → Start accepting requests as Raft leader at block [0] channel=mychannel node=1
2026-08-20 14:28:07.014 UTC 002d WARN [common.deliver] Handle → Error reading from 172.18.0.1:37342: rpc error: code = Canceled desc = context canceled
2026-08-20 14:28:07.014 UTC 002e INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Deliver grpc.peer_address=172.18.0.1:37342 error="rpc
 error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=2.378098ms
2026-08-20 14:28:07.198 UTC 002f WARN [orderer.consensus.etcdraft] checkForEvictionNCertRotation → could not read config metadata: %!s(<nil>) channel=mychannel node=1
2026-08-20 14:28:07.199 UTC 0030 INFO [orderer.consensus.etcdraft] propose → Created block [1], there are 0 blocks in flight channel=mychannel node=1
2026-08-20 14:28:07.199 UTC 0031 INFO [orderer.consensus.etcdraft] run → Received config transaction, pause accepting transaction till it is committed channel=mychannel node=1
2026-08-20 14:28:07.199 UTC 0032 WARN [orderer.common.broadcast] Handle → Error reading from 172.18.0.1:37344: rpc error: code = Canceled desc = context canceled
2026-08-20 14:28:07.199 UTC 0033 INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Broadcast grpc.peer_address=172.18.0.1:37344 error="r
pc error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=4.716671ms
2026-08-20 14:28:07.200 UTC 0034 INFO [orderer.consensus.etcdraft] writeBlock → Writing block [1] (Raft index: 3) to ledger channel=mychannel node=1
2026-08-20 14:28:07.249 UTC 0035 WARN [common.deliver] Handle → Error reading from 172.18.0.1:37346: rpc error: code = Canceled desc = context canceled
2026-08-20 14:28:07.249 UTC 0036 INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Deliver grpc.peer_address=172.18.0.1:37346 error="rpc
 error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=2.289292ms
2026-08-20 14:28:07.395 UTC 0037 WARN [orderer.consensus.etcdraft] checkForEvictionNCertRotation → could not read config metadata: %!s(<nil>) channel=mychannel node=1
2026-08-20 14:28:07.395 UTC 0038 INFO [orderer.consensus.etcdraft] propose → Created block [2], there are 0 blocks in flight channel=mychannel node=1
2026-08-20 14:28:07.395 UTC 0039 INFO [orderer.consensus.etcdraft] run → Received config transaction, pause accepting transaction till it is committed channel=mychannel node=1
2026-08-20 14:28:07.396 UTC 003a WARN [orderer.common.broadcast] Handle → Error reading from 172.18.0.1:37356: rpc error: code = Canceled desc = context canceled
2026-08-20 14:28:07.396 UTC 003b INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Broadcast grpc.peer_address=172.18.0.1:37356 error="r
pc error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=6.075081ms
2026-08-20 14:28:07.397 UTC 003c INFO [orderer.consensus.etcdraft] writeBlock → Writing block [2] (Raft index: 4) to ledger channel=mychannel node=1
2026-08-20 14:29:42.401 UTC 003d INFO [orderer.consensus.etcdraft] propose → Created block [3], there are 0 blocks in flight channel=mychannel node=1
2026-08-20 14:29:42.406 UTC 003e INFO [orderer.consensus.etcdraft] writeBlock → Writing block [3] (Raft index: 5) to ledger channel=mychannel node=1
2026-08-20 14:29:42.430 UTC 003f WARN [orderer.common.broadcast] Handle → Error reading from 172.18.0.1:47242: rpc error: code = Canceled desc = context canceled
2026-08-20 14:29:42.430 UTC 0040 INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Broadcast grpc.peer_address=172.18.0.1:47242 error="r
pc error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=2.039330737s
2026-08-20 14:29:50.602 UTC 0041 INFO [orderer.consensus.etcdraft] propose → Created block [4], there are 0 blocks in flight channel=mychannel node=1
2026-08-20 14:29:50.607 UTC 0042 INFO [orderer.consensus.etcdraft] writeBlock → Writing block [4] (Raft index: 6) to ledger channel=mychannel node=1
```



```
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ docker logs peer0.org1.example.com
2026-08-20 14:27:57.465 UTC 0001 INFO [nodeCmd] serve → Starting peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
2026-08-20 14:27:57.466 UTC 0002 INFO [nodeCmd] serve → Peer config with combined core.yaml settings and environment variable overrides:
chaincode:
  builder: $(DOCKER_NS)/fabric-ccenv:$(TWO_DIGIT_VERSION)
  executetimeout: 300s
  externalbuilders:
  - name: ccaas_builder
    path: /opt/hyperledger/ccaas_builder
    propagateEnvironment:
    - CHAINCODE_AS_A_SERVICE_BUILDER_CONFIG
  golang:
    dynamiclink: false
    runtime: $(DOCKER_NS)/fabric-baseos:$(TWO_DIGIT_VERSION)
  installtimeout: 300s
  java:
    runtime: $(DOCKER_NS)/fabric-javaenv:2.5
  keepalive: 0
  logging:
    format: '%{color}%{time:2006-01-02 15:04:05.000 MST} [%{module}] %{shortfunc}
      → %{level:.4s} %{id:03x}%{color:reset} %{message}'
    level: info
    shim: warning
  mode: net
  node:
    runtime: $(DOCKER_NS)/fabric-nodeenv:2.5
  pull: false
  startuptimeout: 300s
  system:
    _lifecycle: enable
    cscc: enable
    lscc: enable
    qscc: enable
ledger:
  history:
    enablehistorydatabase: true
  pvtdatastore:
    collelgprocdbbatchesinterval: 1000
    collelgprocmaxdbbatchsize: 5000
    deprioritizeddatareconcilerinterval: 60m
  snapshots:
    rootdir: /var/hyperledger/production/snapshots
  state:
```



```
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ ./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1 ...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d4455731128138 89f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
+ res=0
2026-08-20 19:59:35.934 IST 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal → Installed remotely: response:<status:200 payload:"\nJbasic_1.0:1d4d4455731128138 89f87c307bc2b5fbe664
c87b24c035bee15cbcc7f59b629\022\tbasic_1.0" >
2026-08-20 19:59:35.934 IST 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal → Chaincode code package identifier: basic_1.0:1d4d4455731128138 89f87c307bc2b5fbe664c87b24c035bee15cb
c7f59b629
Chaincode is installed on peer0.org1
Install chaincode on peer0.org2 ...
Using organization 2
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d4455731128138 89f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
+ res=0
2026-08-20 19:59:40.264 IST 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal → Installed remotely: response:<status:200 payload:"\nJbasic_1.0:1d4d4455731128138 89f87c307bc2b5fbe664
```

Result

The Hyperledger Fabric network was successfully deployed and managed. The blockchain network components, including peer nodes, ordering service, Certificate Authorities, and application channel, were verified. Chaincode was deployed successfully, network status was monitored, and the Fabric network was shut down and cleaned up successfully.